

LENGUAJE PHP

Ventajas

1. Multiplataforma
2. Abierto y gratuito
3. Gran comunidad de usuarios
4. Apache, MySQL
5. Hitos importantes del lenguaje en PHP 5 (Orientación a objetos) y PHP 7 (mejoras rendimiento y tipado)
6. Extensiones
7. Buen lenguaje para crear aplicaciones web

Herramientas

- Editores de código multipropósito (Sublime, Atom, Notepad++, Visual Studio Code)
- Entornos de desarrollo integrado (IDE):
 -  Basados en **Eclipse**: Aptana, PHP Designer Toolkit (PDT), Zend Studio
 -  **Netbeans**
 -  **IntellyJ Idea** y **PHPStorm**
 -  **Visual Studio**
- Herramientas de desarrollo visual (Adobe Dreamweaver, Microsoft Expression Web)

Variables(1/3)

- Cuando utilizamos el nombre de las variables estamos haciendo **referencia** a un dato que está en **memoria**
- Nombre de la variable=identificador de variable. Tiene que cumplir:
 - ✚ Empezar con \$
 - ✚ El segundo carácter puede ser “_” o una letra (recomendable letra)
 - ✚ A partir del tercer carácter se pueden incluir números además de letras y “_”
 - ✚ No hay límite de tamaño en el nombre
 - ✚ El nombre de la variable no puede tener espacios en blanco (cuidado con mayúsculas y minúsculas)
- PHP no es declarativo (función *isset()* -> devuelve 1 si la variable ha sido declarada, si no, nada):

```
<?php
    $var1="Pepe"; //declaro una variable
    $var1 .= " Pérez"; //añado a mi variable anterior
    echo $var1;
?>
```



```
C:\php\archivos>php variables.php
Pepe Pérez
```

```
<?php
    $var2="2";
    $var2="María"; //Ahora var2 vale María
    $var3= $var2 . "\n" . "Angelines";
    echo "var2 vale: ", $var2, " y var3: ", $var3;
?>
```



```
C:\php\archivos>php variables.php
var2 vale: María y var3: María
Angelines
```

Variables(2/3)

Tipos de datos (1/2)

1. Números enteros

```
$n1=17;      $octal=071;
$n2=-175;    echo $octal; //escribe 56
```

```
$hexa=0xA2BC;
echo $hexa; //escribe 41660
```

2. Números en coma flotante (formato *double* en C)

```
$n1=234.12;
$n2=12.3e-4; //significa 12,3·10-4
```

3. Cadenas o strings

```
$nombre="Jorge Sánchez";
$frase = 'Antonio dijo "Hola" al llegar';
$frase = "Antonio dijo \"Hola\" al llegar";
```

Las secuencias de escape sólo funcionan si están encerradas entre comillas dobles, por lo que en lo siguiente las barras saldrían también por pantalla:

```
echo 'Antonio dijo \"Hola\" al llegar';
```

<code>\t</code>	Tabulador
<code>\n</code>	Nueva línea
<code>\f</code>	Alimentación de página
<code>\r</code>	Retorno de carro
<code>\"</code>	Dobles comillas
<code>'</code>	Comillas simples
<code>\\</code>	Barra inclinada (backslash)
<code>\\$</code>	Símbolo dólar

Variables(3/3)

Tipos de datos (2/2)

4. Booleanos

```
$verdadero=true;
echo $verdadero; //escribe 1
```

- Enteros -> **false**: cero, **true**: resto
- Coma flotante-> **false**: 0.0, **true**: resto
- Cadenas-> **false**: cadena vacía
- Arrays-> **false**: no contiene ningún elemento
- Recursos-> **false**: el recurso no es válido

5. Conversiones

```
$v1=18;
$v2="3 de Diciembre";
echo $v1+$v2; //Escribe 21
```

```
$x=2.5;
$y=4;
$z=(int)$x * $y; //z vale 8
```

Podemos usar los siguientes operadores de casting:

- **(int)** o **(integer)**. Convierte a entero
- **(real)**, **(double)** o **(float)**. Convierte a coma flotante
- **(string)**. Convierte a forma de texto
- **(array)**. Convierte a array.
- **(object)**. Convierte a un objeto

Referencias &

```
$nombre="Antonio";  
$ref=&$nombre; //$ref es una referencia a la variable $nombre  
echo $ref."<br />"; //escribe Antonio  
$ref="Marisa"; //ahora se asigna el valor Marisa, a $ref  
echo $nombre."<br />"  
//escribe Marisa, a través de la referencia se ha cambiado el  
//valor de la variable $nombre
```

Constantes

Almacenan valores que no cambian en el tiempo, **no utilizan el signo \$** de las variables

- **Define** o **const**

```
<?php  
  
define("PI", 3.1416);  
const PI2=3.1415;  
echo PI2;  
  
?>
```

Variables de variables

En PHP está permitido utilizar **macrosustitución de variables**

```
$x="variable1";
$$x=5;
echo $variable1; //escribe 5
```

Útil para conseguir crear una variable de cada elemento de un array

Operadores (1/3)

1. Aritméticos

OPERADOR	SIGNIFICADO
+	Suma
-	Resta
*	Producto
/	División
%	Módulo (resto)

```
$x=15.5;
$y=2;
echo $x+$y,"<br />"; //escribe 17.5
echo $x-$y,"<br />"; // escribe 13.5
echo $x*$y,"<br />"; // escribe 31
echo $x/$y,"<br />"; //escribe 7.75
echo $x%$y,"<br />"; //escribe 1, solo coge la parte entera
```

Operadores (2/3)

2. Condicionales: Sirven para comparar valores, siempre devuelven valores booleanos

OPERADOR	SIGNIFICADO
<	Menor
>	Mayor
>=	Mayor o igual
<=	Menor o igual
==	Igual, devuelve verdadero si las dos expresiones que compara son iguales
===	Equivalente, devuelve verdadero si las dos expresiones que compara son iguales y además del mismo tipo
!=	Distinto
!	No lógico (NOT)
&&	“Y” lógico
AND	“Y” lógico
	“O” lógico
OR	“O” lógico
XOR	“OR” exclusivo

Los operadores lógicos (AND, OR y NOT), sirven para evaluar condiciones complejas. NOT sirve para negar una condición. Ejemplo:

```
$edad = 21;
$mayorDeEdad = $edad >= 18; //$mayorDeEdad será true
$menorDeEdad = !$mayorDeEdad; //$menorDeEdad será false
```

El operador && (AND) sirve para evaluar dos expresiones de modo que, si ambas son ciertas, el resultado será **true** si no el resultado será **false**. Ejemplo:

```
$carnetConducir=true;
$edad=20;
$puedeConducir= ($edad>=18) && $carnetConducir;
//$puedeConducir es verdadero, puesto que $edad es mayor de 18
//y $CARNET es true
```

El operador || (OR) sirve también para evaluar dos expresiones. El resultado será true si al menos uno de las expresiones es true. Ejemplo:

```
$nieva =TRUE;
$llueve=FALSE;
$graniza=FALSE;
$malTiempo= $nieva||$llueve||$graniza;//malTiempo es TRUE porque nieva
```

La diferencia entre la igualdad y la equivalencia se puede explicar con este ejemplo:

```
$resultado= (18 == 18.0); //resultado es verdadero
$resultado= (18 === 18.0); //resultado es falso
```

En el ejemplo `18===18.0` devuelve falso porque aunque es el mismo valor, no es del mismo tipo.

Operadores (3/3)

3. Operadores de asignación: Da valor a una variable

+=	-=	*=	/=
&=	=	^=	%=
>>=	<<=	.=	

```
$x += 3;
```

```
$x++; //equivalente a $x=$x+1;
$x--; //equivalente a $x=$x-1;
```

```
$x=5;
```

```
$y=5;
```

```
$z=$x++; // Primero asigna, luego incrementa: z vale 5, x vale 6
```

```
$z=++$y; // Primero incrementa, luego asigna: z vale 6, x vale 6
```

4. Operadores de bit: Permite utilizar operaciones binarias, aplicados a números, usan su forma binaria para realizar la operación bit a bit

símbolo	significado
&	AND
	OR
~	NOT
^	XOR
>>	Desplazamiento a la derecha
<<	Desplazamiento a la izquierda

```
$x=97;           //$x en binario vale 01100001
$y=74;           //$y en binario vale 01001100
echo $x & $y; //resulta:----->01000000=64
echo "\n";
echo $x | $y; //resulta:----->01101101=107
echo "\n";
$x = $x >> 2; //ahora $x vale 00011000 = 24
echo $x;
echo "\n";
$y = $y << 2; //ahora $y vale 100110000 = 296
echo $y;
```

Estructuras de control: Sentencia condicional if (1/2)

SENTENCIA CONDICIONAL SIMPLE

```
if(expresiónLógica) {
    instrucciones
}
```

```
if($nota>=5){
    echo "Aprobado";
    $aprobados++;
}
```

Las llaves sólo se requieren si va a haber varias instrucciones, si sólo hay una podemos obviarlas

Otra posibilidad de if:

```
if(expresión lógica) :
    instrucciones
...
endif;
```

SENTENCIA CONDICIONAL COMPUESTA

```
if(expresión lógica){
    instrucciones a ejecutar si la expresión es verdadera
    ...
}
else {
    instrucciones a ejecutar si la expresión es falsa
    ...
}
```

```
if($nota>=5):
    echo "Aprobado";
    $aprobados++;
else :
    echo "Suspenso";
    $suspensos++;
endif;
```

Estructuras de control: Sentencia condicional if (2/2)

SENTENCIA CONDICIONAL MÚLTIPLE

```
if($nota<5){
    echo "Suspenso";
    $suspensos++;
}
elseif($nota<7){
    echo "Aprobado";
    $aprobados++;
}
elseif($nota<9){
    echo "Notable";
    $aprobados++;
}
else{
    echo "Sobresaliente";
    $aprobados++;
}
```

```
if($nota<5):
    echo "suspenso";
    $suspensos++;
elseif($nota<7):
    echo "aprobado";
    $aprobados++;
elseif($nota<9):
    echo "notable";
    $aprobados++;
else:
    echo "sobresaliente";
    $aprobados++;
}
```

Estructuras de control: Sentencia Switch

- Condicional, similar a if

```
switch ($diaSemana) {
    case 1:
        $dia="Lunes";
        break;
    case 2:
        $dia="Martes";
        break;
    case 3:
        $dia="Miércoles";
        break;
    case 4:
        $dia="Jueves";
        break;
    case 5:
        $dia="Viernes";
        break;
    case 6:
        $dia="Sábado";
        break;
    case 7:
        $dia="Domingo";
        break;
    default:
        $dia="?";
}
```

¿break, continue?

default y *break* son opcionales

default: en caso de que no se cumpla ninguna de las anteriores, deberían cumplirse esas instrucciones

break: una vez que se cumpla dicha condición, salir del switch

```
switch (variable)
{
    case 'value':
        # code...
        break;

    default:
        # code...
        break;
}
```

Estructuras de control: Bucles (1/3)

Un bucle es un conjunto de sentencias que se repiten mientras se cumpla una determinada condición.

BUCLE WHILE

Se evalúa la expresión lógica, si es verdadera ejecuta las sentencias, si no el programa abandona la sentencia while y tras ejecutar la sentencia se repite el proceso

```
while (expresión_lógica) {
    sentencias que se ejecutan si la condición es true
}
```

```
$i=1;
while ($i<=100){
    echo $i."<br />";
    $i++;
}
```

```
$i=1;
while ($i<=100):
    echo $i."<br />";
    $i++;
endwhile;
```

1. **Bucle de contador:** se repiten un número concreto de veces, es seguro que finalice
2. **Bucle de centinela:** no se tienen que repetir un número concreto de veces, no es seguro que finalice

Puede haber bucles que mezclen ambos tipos

Estructuras de control: Bucles (2/3)

BUCLE DO...WHILE

El bucle se ejecuta al menos una vez. Primero ejecuta las sentencias y luego evalúa la expresión lógica, si la expresión es verdadera vuelve a ejecutar las sentencias, si no sale del while

```
do {
    instrucciones
} while (expresión lógica)
```

```
$i=0;
do {
    $i++;
    echo $i."<br />";
} while ($i<100);
```

Siempre podemos convertir un bucle do...while en while
(tendríamos que poner fuera las instrucciones una vez)

BUCLE FOR (1/2)

Pensado para recorrer arrays o para implementar bucles de contador

```
for(inicialización;condición;incremento){
    sentencias
}
```

```
for(inicialización;condición;incremento):
    sentencias
endfor;
```

Estructuras de control: Bucles (3/3)

BUCLE FOR (2/2)

Su funcionamiento es el siguiente:

1. Se ejecuta la instrucción de inicialización
2. Se comprueba la condición
3. Si la condición es cierta, entonces se ejecutan las sentencias. Si la condición es falsa, se abandona for
4. Tras ejecutar las sentencias, se ejecuta la instrucción de incremento y se vuelve al paso 2

El número de líneas de código se reduce, pero el código es menos comprensible

Son equivalentes los siguientes bucles:

```
for($i=1;$i<=1000;$i++){  
    echo $i."<br />";  
}
```

```
$i=1; // sentencia de inicialización  
while($i<=1000) { // condición  
    echo $i."<br />";  
    $i++; // incremento  
}
```

Redirigir hacia otra página

Función header:

Gracias a ella, podemos utilizar la cabecera **Location** del protocolo http que provocará que se cargue otra página

```
header("Location: http://www.google.es");
```

